

Security Audit

Moonopol (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	13
Audit Resources	13
Dependencies	13
Severity Definitions	14
Status Definitions	15
Audit Findings	16
Centralisation	25
Conclusion	26
Our Methodology	27
Disclaimers	29
About Hashlock	30

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Moonopol team partnered with Hashlock to conduct a security audit of their moonopol-alpha Solana program. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Moonopol-alpha is a Solana program implemented with Anchor that creates and manages decentralized binary prediction markets, enabling users to place and match both limit and market orders using USDC as collateral. It handles market initialization, order matching via complex PDA-based mechanisms, market resolution, and rewards distribution while maintaining strict economic and state invariants.

Project Name: Moonopol

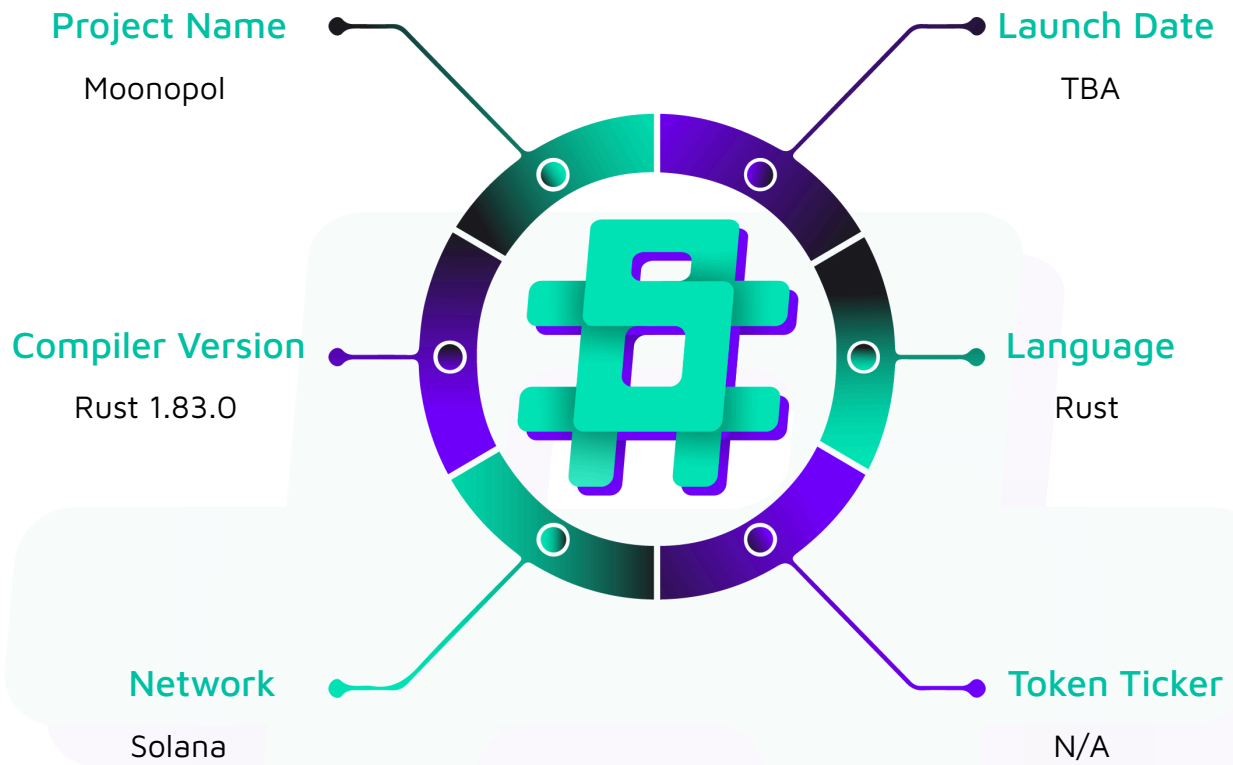
Project Type: Defi, prediction market

Compiler Version: Rust 1.83.0

Website: <https://moonopol.com/>

Logo:



Visualised Context:

Project Visuals:



Expect us.



Audit Scope

We at Hashlock audited the Solana Rust code within the Moonopol project, the scope of work included a comprehensive review of the programs listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Moonopol programs
Platform	Solana
Audit Date	February, 2025
Contract 1	errors.rs
Contract 1 MD5 Hash	7ba843d2eef221e7bd246746c2657bb8
Contract 2	lib.rs
Contract 2 MD5 Hash	7188c18b1a0d4d843b8c8bf22f9eb9d3
Contract 3	constants.rs
Contract 3 MD5 Hash	e731ec41c9c2a04858c9822cb9743302
Contract 4	order_matcher.rs
Contract 4 MD5 Hash	b6e3c7ba0f693049e7f075b230c4a6df
Contract 5	order_placer.rs
Contract 5 MD5 Hash	6e07869c1819c241fe5fcdade2af1d752
Contract 6	cancel_limit_order.rs
Contract 6 MD5 Hash	60f6f11023d5fdf1f8c4e36cec2e0822
Contract 7	claim_rewards.rs
Contract 7 MD5 Hash	6df719ac77bdd7c116953b5fa0124ba5
Contract 8	initialize_market.rs
Contract 8 MD5 Hash	9eb3d8f02415ac89cd2ce8a36ebb02c0

Contract 9	match_orders.rs
Contract 9 MD5 Hash	ff1a821d346e72f7a8ed0424b5d2a55d
Contract 10	place_and_match_limit_order.rs
Contract 10 MD5 Hash	a1a5678ead17b3ee8be20dc736afd34e
Contract 11	place_limit_order.rs
Contract 11 MD5 Hash	914bc29fe60ad297135bfc9093dc19b3
Contract 12	place_market_order.rs
Contract 12 MD5 Hash	b32926c315b1d62ed49a0de622355723
Contract 13	resolve_market.rs
Contract 13 MD5 Hash	2689025aeb852c7538b453bfa243c009

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow advanced logic, with complex operations and assumptions. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

2 Medium severity vulnerabilities

4 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
constants.rs Provides global parameters, defining constants such as the order limit and fee basis points that are used across the program's logic.	File achieves this functionality.
errors.rs Standardizes error handling by defining a set of custom error codes and messages that encapsulate the various failure states encountered during market operations.	File achieves this functionality.
instructions/cancel_limit_order.rs Enables users to cancel an active limit order from their user market state, refunding collateral for buy orders and removing the order from the order book.	While it fulfills its functional purpose, the file's logic is undermined by insufficient validation of the market account used to derive the market_authority PDA, which could allow unauthorized token transfers.
instructions/claim_rewards.rs Permits users to claim rewards accrued from their participation in a resolved market by calculating reward amounts based on share holdings, transferring tokens from the market vault to the user's token account, and then attempting to close the user market state account.	File achieves this functionality.
instructions/initialize_market.rs Allows an authorized actor to initialize a new market by	File achieves this functionality.

setting market metadata, resolution time, and the initial state for vaults and PDAs required for subsequent trading and settlement activities.	
instructions/match_orders.rs Facilitates the matching of maker and taker orders by executing trade logic that includes price verification, fee calculations, state updates, and the emission of order events. It also contains specific branches for self trades and cross-side orders.	Although the core functionality is achieved, the file's order matching logic suffers from potential issues in self trade handling and remaining_accounts verification that could lead to state inconsistencies or unauthorized account manipulation.
instructions/place_and_match_limit_order.rs Allows users to submit limit orders that are immediately processed against existing orders in the order book. If the order is only partially filled, the remainder is reinserted into the order book as a new order.	The file successfully executes the combined placement and matching process, but the re-assignment of order identifiers and reliance on remaining_accounts ordering present potential edge-case issues.
instructions/place_limit_order.rs Enables users to place a limit order directly into the market's order book without immediate matching, ensuring that, in the case of a buy order, the corresponding USDC is transferred into the market vault as collateral.	File achieves this functionality.

instructions/place_market_order.rs Permits users to execute market orders that immediately engage available maker orders, incorporating mechanisms for slippage tolerance, partial fills, token transfers, and refunding any unspent collateral.	The file largely meets its functionality, although issues such as a division-by-zero risk at maximum price and an unused slippage parameter compromise the intended price execution safeguards.
instructions/resolve_market.rs Allows an authorized party to resolve a market by specifying the outcome, transferring accumulated fees from the market vault to a designated fee vault, and updating the market state to reflect its resolved status.	File achieves this functionality.
state/order_matcher.rs Implements the core logic for matching orders, including determining execution prices, computing filled sizes, transferring tokens using CPIs, and handling various trade scenarios such as same-side and different-side orders as well as self trades.	File achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Moonopol project, as outlined in the Audit Scope section. The prediction market logic uses advanced logic utilizing multiple types of matching, rewarding and decision-making. Some refactoring was required to mitigate the likelihood of exploitation.

Audit Resources

We were given the Moonopol project smart contract code in the form of Github access.

As mentioned above, code parts are mostly not commented on. The logic is complex, and therefore it is hard to quickly comprehend the programming flow as well as the complex code logic.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] moonopol#increment_shares - Development-only function exposure

Description

The `increment_shares` function, intended for development use, lacks proper feature-gating in release builds. The commented-out `#[cfg(feature = "dev")]` macro fails to restrict access, leaving the function callable in production.

Vulnerability Details

The protocol includes a function originally intended for development purposes, that is `increment_shares`, which lacks protection in production builds due to a commented-out development macro. This oversight means that the function remains callable in release mode, exposing the contract to unauthorized state manipulation.

Impact

An attacker could exploit this function to arbitrarily adjust share balances, thereby manipulating market outcomes and reward calculations without incurring any economic cost.

Recommendation

Protect the function by using conditional compilation flags that exclude it from production builds or remove it entirely from release code to ensure that only authorized and intended operations can alter share balances.

Status

Resolved

[H-02] moonopol#match_order - Self trade handling enables economic abuse

Description

Self-trades (orders by the same user) bypass economic checks, allowing users to inflate `yes_shares` and `no_shares` without cost when shares are sold to themselves.

Vulnerability Details

The order matching process contains a branch that allows self trades, where orders submitted by the same owner are matched against each other without imposing any economic cost. The logic merely increments share counts on both sides of the trade, permitting a single actor to artificially inflate their positions, due to the missing decrease logic when they are sold to themselves.

Impact

This vulnerability enables an attacker to execute trades against themselves to inflate their share positions and claim disproportionate rewards without any corresponding financial risk, thereby destabilizing market fairness and enabling potential profit extraction through internal manipulation.

Recommendation

Rework the self trade handling to either disallow self trades entirely or implement the correct calculations of shares after they are sold.

Status

Resolved

[H-03] moonopol#close_market - Incomplete market closing logic leaving funds at risk

Description

The `close_market` function sets a `closed` flag but does not refund user deposits or cancel open orders. .

Vulnerability Details

The `close_market` instruction simply sets a `closed` flag within the market state without executing any additional procedures to return or unlock the funds held in the market vault. No mechanism is in place to ensure that the funds are retrievable once the market is closed.

Impact

This omission leaves user funds locked within the vault indefinitely and creates a scenario where assets become inaccessible.

Recommendation

Develop and integrate a complex market closure mechanism that securely transfers or unlocks funds from the vault, ensuring that users can recover their assets and that the protocol state remains consistent and transparent.

Status

Resolved

[H-04] moonopol#place_market_order - Slippage parameter ignored in price calculations

Description

The `slippage_percent` parameter is validated but unused during price calculations.

Vulnerability Details

Although the `place_market_order` instruction validates the `slippage_percent` parameter, this value is not integrated into the subsequent price calculations. Consequently, the expected tolerance for execution prices is effectively disregarded.

Impact

Users may receive trade executions at prices that exceed their specified slippage tolerance, exposing them to potential financial losses and undermining the trust in the protocol's order matching system.

Recommendation

Modify the order matching logic to incorporate the `slippage_percent` parameter directly into the price execution calculations.

Note

The Moonopol team informed Hashlock that the recommendation will be implemented in the near future.

Status

Acknowledged

Medium

[M-01] moonopol#place_market_order - Division-by-zero in price execution calculation

Description

Maker orders at MAX_PRICE (1,000,000) set execution_price to zero, causing division errors.

Vulnerability Details

In the place_market_order instruction, when a maker order is submitted at the maximum allowed price (1,000,000), the calculated effective price for opposing orders becomes zero. This zero value then causes a division-by-zero error during the computation of execution sizes, effectively bypassing the matching process for such orders.

Impact

The division-by-zero condition not only results in the matching loop skipping processing but may also cause transaction failures or create exploitable edge cases that could be manipulated to disrupt the market's order-matching functionality.

Recommendation

Implement additional safeguards that validate the effective price to ensure it is non-zero before any division operations occur, and adjust the matching logic to handle maximum price edge cases appropriately without resorting to a zero effective price.

Status

Resolved

[M-02] moonopol#execute - Ignored errors in taker order removal post-match**Description**

Errors during taker order removal are silently ignored, leaving stale orders in user accounts.

Vulnerability Details

In the `execute` function, in line 112, errors that occur during the removal of the taker order are deliberately ignored. This practice can lead to scenarios where stale orders remain in the user's `UserMarketState.orders` array even after a match is processed.

Impact

The persistence of stale orders may eventually break the user's order array, triggering a `UserMarketAccountFull` error that prevents the placement of new orders. An attacker could exploit this mechanism by repeatedly initiating matches that leave behind orphaned orders, effectively performing a denial of service against the user's ability to trade.

Recommendation

Ensure that any errors encountered during the removal of the taker order are properly propagated and handled. The removal process must be atomic, with comprehensive error checking to maintain the integrity and consistency of the user's order state.

Status

Resolved

Low

[L-01] moonopol#errors - Duplicate error codes with redundant messages

Description

The protocol defines error codes `MarketAlreadyResolved` and `MarketResolved` with identical error messages and semantic meaning, resulting in redundancy within the error handling framework.

While this redundancy does not directly lead to a security breach, it may cause confusion during debugging and further development, and could lead to inconsistent error handling in the codebase.

Recommendation

Consolidate the error codes by removing the redundant one and ensure that error handling throughout the protocol uses a clear, singular, and precise error identifier for resolved market states.

Status

Resolved

[L-02] moonopol#resolve_market - Omission of fee aggregation in market resolution

Description

In the `resolve_market` instruction, the computed `fee_amount` is not added to the `market.total_fees` state variable. This discrepancy between fee calculation and state update means that the recorded fee totals will not accurately reflect the fees collected during market resolution.

An inaccurate fee total compromises audit trails and can lead to misallocation of funds during fee distribution, potentially affecting future state verifications.

Recommendation

Correct the market resolution logic to ensure that the computed `fee_amount` is properly aggregated into `market.total_fees`.

Status

Resolved

[L-03] moonopol#claim_rewards - Incomplete closure of user market state in `claim_rewards`

Description

The `close_user_market_state` function in the `claim_rewards` instruction manually transfers lamports from the `user_market_state` to the user account without invoking Anchor's automatic account closure mechanism. This leaves the account allocated with residual state despite its lamport balance being reduced to zero.

An account that remains open despite being functionally "closed" can be reinitialized or exploited for unintended purposes.

Recommendation

Replace the manual lamport transfer with the use of Anchor's `close` attribute to ensure that the `user_market_state` account is fully deallocated and its state cleared, thereby preventing any further misuse or reinitialization.

Status

Resolved

[L-04] moonopol#match_order - Redundant second execution of `is_self_trade` in order matching

Description

Within the order matching logic, the function `is_self_trade` is executed in `else if` second time unnecessarily. This redundant invocation does not contribute additional security or validation and merely complicates the logic flow.

Although the redundant call does not directly expose a vulnerability, it introduces unnecessary computational overhead and potential confusion in the matching process, which could lead to maintainability issues over time.

Recommendation

Remove the redundant invocation to streamline the order matching logic and improve overall code clarity and efficiency without altering the protocol's behavior.

Status

Resolved

Centralisation

The Moonopol project values decentralisation, mitigates the risk of potentially malicious owners compromising the security of the solution as a whole.



Conclusion

After Hashlock's analysis, the Moonopol project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code needs to be commented properly. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.